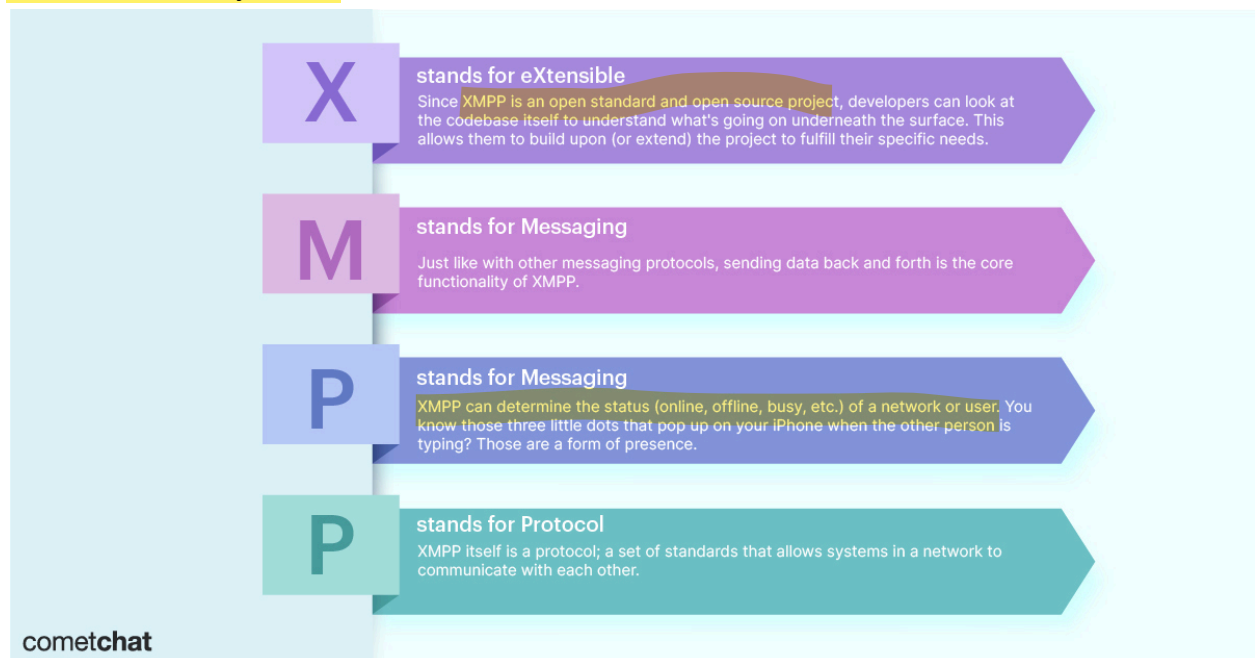


XMPP

The acronym XMPP stands for “Extensible Messaging and Presence Protocol”, which refers to an extensible messaging and presence protocol for online communication.

XMPP’s principles are the client-server principle and the open XML standard, which allow an XMPP client on the user’s terminal to communicate with other participants via an XMPP server.

At its root, XMPP is a chat protocol that allows for the transmission of XML fragments, i.e. data, between two network endpoints in near real-time. Unlike other chat protocols, the XMPP protocol is designed for small chunks of XML data rather than large blobs or streams of binary data.



With these components, XMPP can handle many of the core functions of a chat app including:

1. Sending and receiving messages
2. Broadcasting a user's status
3. Managing a friends/contact list
4. And my personal favorite—blocking users from sending you messages

How Does XMPP Chat Work?

XMPP works on a client-server architecture. This means that when you send a message via XMPP, it's first sent to a server which then routes it to the correct client (user).

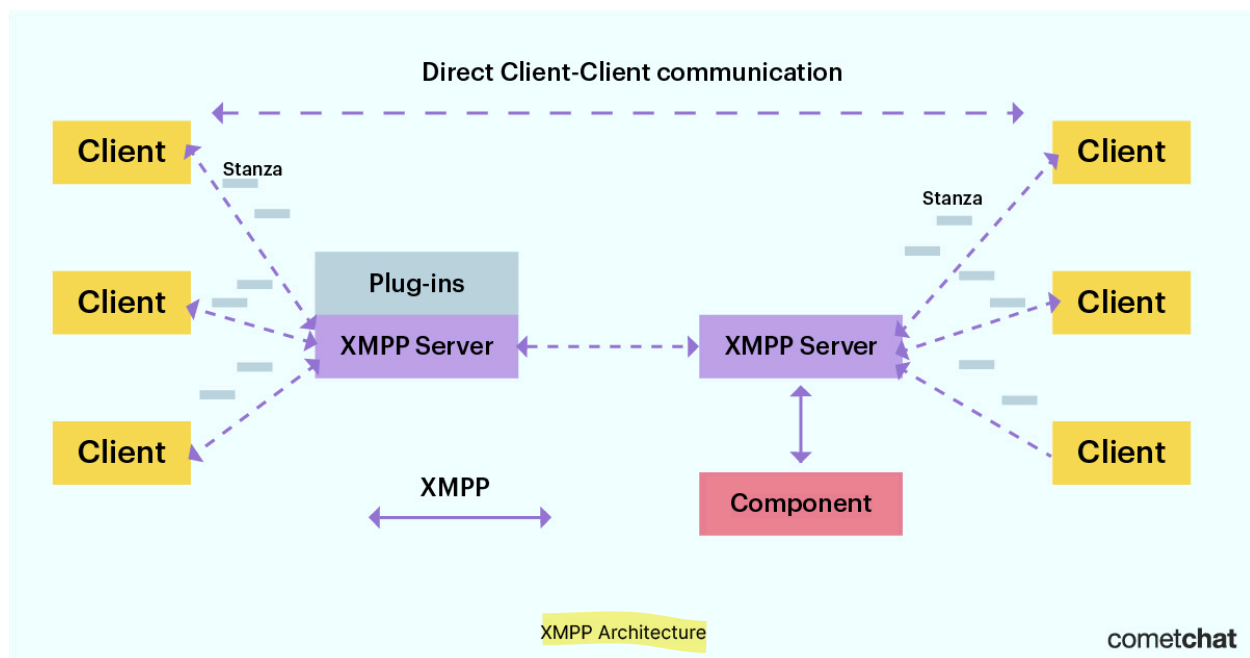
How does it know which client to send your message to?

Well, each client connected to an XMPP server is assigned a unique identifier, known as a **Jabber ID**. The current format of the unique identifier is `user@domain.com/resource`. **User** is the username of the person and `domain.com` is the **domain** of the client that's sending the message.

Resource refers to the type of **device** on which the message was sent. For example, mobile vs web. This part of the unique identifier is optional and is often only defined in situations where the client doesn't support every device.

Using this unique identifier, an XMPP server can route each message to the correct client. When a client initiates a session with an XMPP server, it opens a persistent TCP connection and starts an **XML stream to the server**. Once the client is identified by the server (via the unique identifier) and the connection is accepted, the server then opens an additional XML stream—with this stream going back to the client. The end result is a bi-directional stream of XML data.

XMPP servers can also communicate with other XMPP servers to create a global messaging network called a "federation". A federation allows you to communicate seamlessly with a client on a different XMPP server.



Now that you're familiar with the XMPP architecture at a high level, let's break it down a little further:

XMPP Stanzas

As we mentioned earlier, XMPP transmits fragments of XML between two network endpoints. Specifically, the XML fragments transmitted by XMPP, and used for basic communication, are called stanzas. There are 3 types of stanzas used in XMPP and each have their own unique structure and purpose. These are the presence, message, and IQ stanzas.

Presence stanza

Just as the name suggests, the presence stanza is used to share the **status** of a network or user to other users, networks, or servers.

Message stanza

the message stanza is used to share chat messages between users. It's a "push" mechanism that sends information to other XMPP entities without requiring a response. A message stanza has a type attribute that can have a few different values: chat, normal, group chat, headline, and error.

IQ stanza

An IQ stanza is XMPP's method of requesting and modifying data from client to server. It's similar to HTTP's GET and POST request. Each IQ request stanza has an ID value that associates the request with the response stanza.

XMPP Features

Asynchronous Messaging

XMPP allows for asynchronous messaging between users' devices. What does this mean for your app? It means that your users can send multiple messages in a row rather than sending one and needing to wait for a response from the recipient before sending another. It also means that two users don't need to be online at the same time to exchange messages.

With other client-server architectures, the user's (client) device must periodically ping the server to check for new messages. This process is called polling and it detracts from the real-time experience of the user. It can also be quite expensive bandwidth-wise.

Asynchronous messaging gives the user a real-time experience while being efficient and requiring low bandwidth.

Persistent Connection

XMPP works by creating a persistent TCP (Transmission Control Protocol) connection between a client and server or between one server and another server. Because the connection is persistent, data can be transmitted without reestablishing a connection every time. The low number of connections required reduces bandwidth costs, minimizes network congestion, lowers latency, and lessens CPU and memory usage due to the lower number of connections required.

Decentralization

An often overlooked feature of XMPP is that it's decentralized. You're not reliant on any proprietary server managed by a private company.

For example:

Email: A user with `alice@gmail.com` can send an email to `bob@yahoo.com` because both Gmail and Yahoo follow the same **SMTP** protocol for email exchange.

Federated Chat: A user on `alice@xmppserver.com` can chat with `bob@anotherxmppserver.net` because both servers follow the **XMPP** or **Matrix** protocol for messaging.

Gateways

A key feature of XMPP's design is that it can be used to connect multiple types of protocols such as XMPP -> SMS or SMS -> XMPP -> email. This is done using something called gateways.

Gateways are special services that run alongside an XMPP server and act as client proxies. This server-side aspect of gateways means that the client can allow its users to interact with other users on different protocols—no additional code required.

Ref: <https://www.cometchat.com/blog/xmpp-extensible-messaging-presence-protocol>

WebSockets

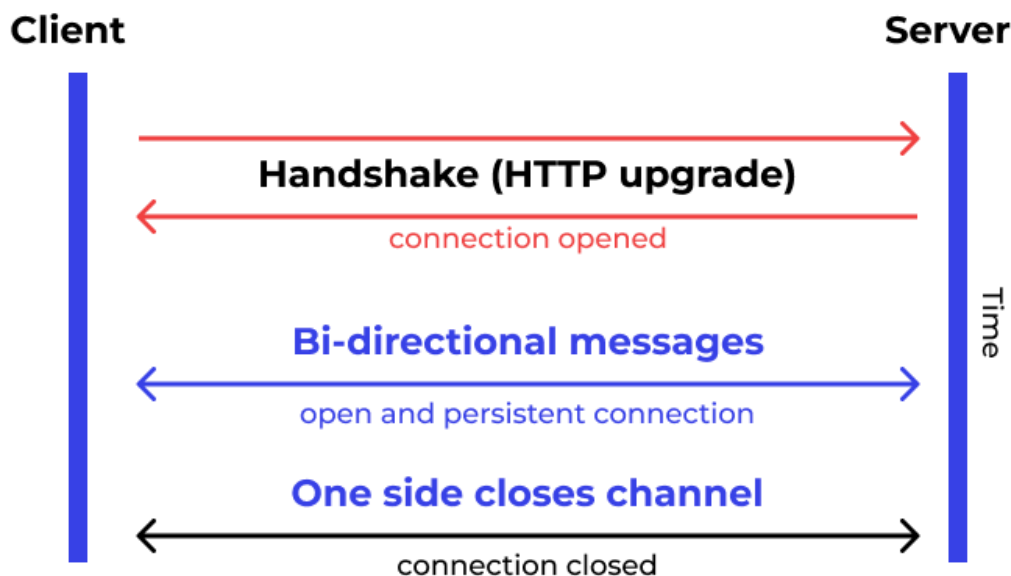
A WebSocket is a **persistent connection** between a client and server. WebSockets provide a **bidirectional, full-duplex** communications channel that **operates over HTTP through a single TCP/IP socket connection**. At its core, the WebSocket protocol facilitates message passing between a client and server.

To better explain, WebSockets send messages between client in real-time when a connection is created and also interacts with the server to fetch information if required.

How do WebSockets work ?

At a high level, working with WebSockets involves three main steps:

- **Opening a WebSocket connection.** The process of establishing a WebSocket connection is known as the opening handshake, and consists of an HTTP request/response exchange between the client and the server.
- **Data transmission over WebSockets.** After a successful opening handshake, the client and server can exchange messages (frames) over the persistent WebSocket connection. WebSocket messages may contain string (plain text) or binary data.
- **Closing a WebSocket connection.** The connection, developed using the WebSocket, lasts as long as any of the participating parties lays it off. Once one party breaks the connection, the second party won't be able to communicate as the connection breaks automatically at its front. Both the client and the server can initiate the closing handshake by sending a close message.



When not to use WebSocket?

WebSocket can be used if we want any real-time updated or continuous streams of data that are being transmitted over the network but if we want to fetch old data, or want to get the data only once to process it with an application we should go with HTTP protocol, old data which is not required very frequently or fetched only once can be queried by the simple HTTP request, so in this scenario, it's better not use WebSocket.

Use cases of WebSocket

1. Real-time updates web applications
2. Chat applications
3. Multi-player games

WebSocket Vs HTTP

Connection

The HTTP protocol is unidirectional protocol works on the top of TCP protocol which is a connection-oriented transport layer protocol, we can create the connection by using HTTP request methods after getting the response HTTP connection get closed.

WebSocket is a bidirectional communication protocol that can send the data from the client to the server or from the server to the client by reusing the established connection channel. The connection is kept alive until terminated by either the client or the server.

Performance

WebSockets leverage on the use of real-time update to push data from the server to the client to be consumed by the user making it very fast and performant and is used by large applications with huge users base that requires information kept up to date without leaving our the performance of the application. Performance is also certified by WebSockets because when a connection is created it remains that way.

HTTP if properly used, can maximize performance. HTTP requires for connection every time it hits the server and that to a certain extent hinders performance in applications.

High-Frequency Messaging

The WebSocket protocol offers a persistent connection to exchange messages. This means that individual messages don't incur any additional tax to establish the transport. Taxes such as establishing SSL, content negotiation, and exchange of bulky headers are imposed only once when the connection is established. **There is virtually no tax per message.**

HTTP may allow multiple requests to reuse a single connection, there will generally be small timeout periods intended to control resource consumption.

Ref: <https://www.cometchat.com/tutorials/what-is-websockets>
<https://ably.com/topic/websockets#how-do-web-sockets-work>
<https://www.wallarm.com/what/a-simple-explanation-of-what-a-websocket-is>

WebRTC (Web Real-Time Communication)

WebRTC is a free, open framework for the web that enables the **real-time communication (RTC)** its name promises to deliver. WebRTC is an **HTML5** standard **JavaScript** allows you to control the **camera and microphone in the browser** with no **downloads**.

It allows you to set up a **peer-to-peer connection** between two different browsers or more to **exchange video and audio** between them.

It enables **real-time voice, text, and video streaming** between web browsers and devices.

Inherently Low Latency. WebRTC **knocks it out of the park** when it comes to **delivery speed**. At sub-500-millisecond glass-to-glass latency, WebRTC offers the quickest method for transporting video across the internet.

Platform and Device Independence. All major browsers and devices support WebRTC, making it simple to integrate into a wide range of apps without dedicated infrastructure.

Adapts to Network Conditions. WebRTC **ensures reliable publishing over poor network conditions** with adaptive network encoding. Your **device sends multiple versions** of the same video stream, each at a different quality and bitrate (like low, medium, and high quality). The receiver (or media server) **automatically picks** the best quality stream based on network conditions.

How WebRTC establishes a peer to peer connection

Link: [P2P connection with WebRTC](#)

📺 What are STUN and TURN Servers? (WebRTC Tips from WebRTC.ventures)

Ref: <https://getstream.io/glossary/webrtc-protocol/>

<https://www.wowza.com/blog/what-is-webrtc>

<https://chatgpt.com/share/67d14add-1c80-800d-90d6-abb7b6004a69>